

ARCHITECTURE

How the WebCV site works, and why

Sudarshana Chaitanya B N

Last updated: August 13, 2026

Why it matters

This explains the WebCV project well enough to walk someone else — a teammate, a reviewer, an interviewer — through it without relying on memory. It covers what each piece does, why it was built that way instead of some other way, and what actually broke along the way. **SPEC.md** is the requirements doc (what the site must do); **PLAN.md** is the execution tracker (what's done, what's open); this document is the “how it's built, and why.”

Contents

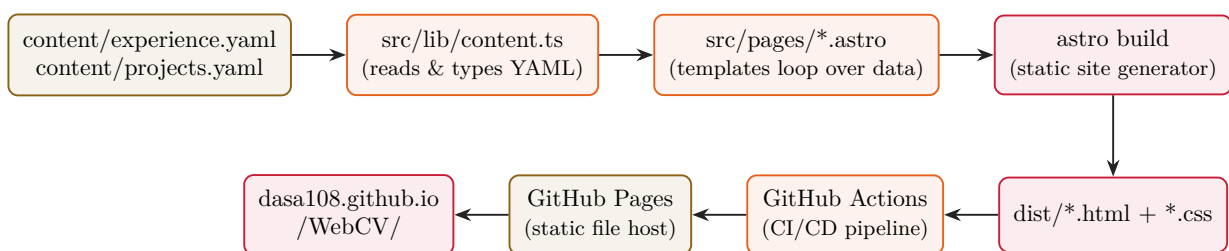
1	The 30-Second Pitch	2
2	Big Picture: From a YAML File to a Live URL	2
3	Why Astro? What a Static Site Generator Is	2
3.1	The Three Real Options Weighed	2
4	The Content Model: Why content/ Is Separate From src/	3
4.1	How It's Implemented	3
4.2	Why YAML, Not a Database or a CMS?	3
4.3	Why Not Astro's Built-In “Content Collections”?	4
5	Repository Tour: What Every File Does, and Why It's There	4
5.1	content/ vs. public/: Why Both Exist	7
6	Code Walkthrough	7
6.1	src/lib/content.ts — the Data Layer	7
6.2	src/components/ — Layout, Nav, Footer, Cards	7
6.3	src/pages/ — File-Based Routing and the Dynamic-Route Trick	8
6.4	src/styles/global.css — Design Tokens, Dark Mode, Responsiveness	8
7	CI/CD: How a git push Becomes a Live Website	9
8	Two Real Bugs Hit and Fixed	10
8.1	Bug 1: Broken Internal Links (/WebCVexperience)	10
8.2	Bug 2: Report/Slide PDF Links Pointed at Files That Don't Exist in the Deployed Site	10
9	Design-Process Decisions Worth Explaining	10
10	Glossary	11
11	Interview Cheat-Sheet	11

1 The 30-Second Pitch

“It’s a personal portfolio site built with Astro, a static site generator. Content — CV, work experience, and projects — lives as plain YAML/Markdown files, completely separate from the page code. At build time, Astro reads that data and pre-renders every page to plain HTML/CSS, which gets deployed to GitHub Pages via a GitHub Actions pipeline that runs automatically on every push. Because content and code are separate, adding a new project or job later means editing one YAML file — no HTML, no redesign, no redeploy button to remember to press.”

That one paragraph answers “walk me through your project” in one breath, and every section below expands on a piece of it.

2 Big Picture: From a YAML File to a Live URL



Two things to internalize, because they come up in almost every follow-up question:

1. **Everything happens at build time, not request time.** There is no server running when someone visits the site. `astro build` runs once, in GitHub’s CI runner — not in the visitor’s browser — and produces plain HTML/CSS files, which is what gets hosted. This is **static site generation (SSG)**.
2. **Content and code are two layers that only meet at build time.** `content/*.yaml` doesn’t know anything about HTML. `src/pages/*.astro` doesn’t know anything about *this specific* internship or project — it only knows how to render *any* entry matching the shape defined in `src/lib/content.ts`. That separation is the whole reason adding a project is a one-file edit instead of a code change (§4).

3 Why Astro? What a Static Site Generator Is

The problem to solve: almost all of this site’s content changes rarely (a CV doesn’t change daily), and there’s no login, no database, no server-side logic — it’s fundamentally a document, not an application. Serving a full client-rendered JavaScript app for that solves a problem that doesn’t exist here, at the cost of a slower first load and more moving parts.

3.1 The Three Real Options Weighed

Option	What it is	Why not chosen
Plain HTML/CSS/JS, no build step	Hand-write every page	No templating — adding a project means copy-pasting a whole card’s HTML by hand, every time. Doesn’t scale.

Option	What it is	Why not chosen
React / Next.js	Full JS framework, client + server rendering	Overkill for a mostly-static content site; ships a JS framework runtime to the browser for pages that don't need inter-activity.
Astro (chosen)	Static site generator with component templating	Renders to plain HTML at build time (fast, simple hosting) but still gives reusable components and data loops — templating power without shipping a framework runtime to the browser.

What “static site generator” means, concretely: a program that takes templates plus data and produces `.html` files *before* anyone visits the site, instead of generating HTML on-the-fly per visitor (server-side rendering, SSR) or in the visitor's browser with JavaScript (a single-page app). Astro's default mode — used here explicitly via `output: 'static'` in `astro.config.mjs` — is SSG: build once, serve static files forever, until the next build.

Why it matters

Static files are the simplest, cheapest, fastest thing to host — a CDN can cache them forever, there's no server to crash or patch, and GitHub Pages hosts them for free. The tradeoff (rebuild to see new content) is a non-issue because publishing a new build is automatic (§7) — it's just a `git push`.

4 The Content Model: Why content/ Is Separate From src/

This is the single most important design decision in the project — **separation of concerns**: data kept separate from presentation. It's a general software design principle, not an Astro-specific trick.

The requirement: adding a new internship or project later should mean editing *one file*, never touching page templates or CSS.

4.1 How It's Implemented

- `content/experience.yaml` and `content/projects.yaml` each hold a flat YAML *list* of entries. Each entry is a plain object — a role, an organization, bullet points, tags. Nothing in these files knows about HTML, colors, or layout.
- `src/lib/content.ts` is the *only* code that touches these files. It reads them with Node's filesystem API and parses them with the `js-yaml` library, exposing typed functions: `getExperience()`, `getProjects()`, `getProjectBySlug(slug)`.
- The page templates (`src/pages/*.astro`) call those functions and loop over whatever comes back — they render “however many entries exist,” never “these specific *N* entries.”

So the flow for “add a new project” is: open `projects.yaml`, copy the `TEMPLATE` comment block at the top, fill in the fields, paste it into the list, commit, push. The Projects page shows one more card automatically, with zero code changes.

4.2 Why YAML, Not a Database or a CMS?

- A database needs a server to run and a network call to query — overkill for a few dozen KB of text that changes maybe once a month.

- A headless CMS (Contentful, Sanity) adds an external account, an API key, and a network dependency for content that’s really just plain text files. YAML in the same git repo means the content is version-controlled, diffable, and needs no third-party service.
- YAML specifically, over JSON, because it supports comments — which is what makes the in-file **TEMPLATE** block and field-by-field explanations possible. That self-documenting schema is what makes it usable by someone (future me) who doesn’t remember the exact field names months later.

4.3 Why Not Astro’s Built-In “Content Collections”?

Content Collections expect one file per entry, living inside `src/content/`. The existing “one list file per type” shape was kept deliberately instead, for two reasons: (1) it’s simpler to reason about — “everything is in this one file” vs. “a new file per entry plus a schema file” — and (2) the whole point of a scratch-built loader (`content.ts`) is full control: no framework magic between the YAML and what actually renders, which keeps debugging a matter of a single function call you can log.

5 Repository Tour: What Every File Does, and Why It’s There

The table below covers every path in the repository. The three-column shape mirrors the question worth asking of any file in a project: what is it, and why does it need to exist at all (as opposed to being deleted or merged into something else)?

Path	What it is	Why it’s needed
<code>SPEC.md</code>	Requirements doc: page structure, color palette, responsiveness/ally targets, acceptance criteria.	Separates “what must this do” from “how do I build it,” and records decisions (breakpoints, palette) that would otherwise live only in memory and drift.
<code>PLAN.md</code>	Execution tracker: what’s done, what’s open, the decision log, the playbook for adding content later.	<code>SPEC.md</code> barely changes; this file changes constantly and records <i>why</i> a choice was made once, so it’s not re-debated in every future conversation about the project.
<code>ARCHITECTURE.md</code> / <code>ARCHITECTURE.tex</code> (this doc)	How the code works and why — tech-stack decisions, code walkthrough, CI/CD, real bugs hit and fixed.	Lets the project be explained to someone else without re-deriving every decision from scratch.
<code>README.md</code>	Entry point: what the project is, links to the other docs, how to run it locally.	The first thing anyone (including future me) reads — has to orient a stranger in under a minute.
<code>package.json</code>	Declares dependencies (<code>astro</code> , <code>js-yaml</code>) and the <code>npm run {dev,build,preview}</code> scripts.	The standard Node.js manifest — without it, neither a human nor CI knows what to install or how to build.

Path	What it is	Why it's needed
<code>package-lock.json</code>	Exact, pinned dependency version tree.	Makes builds reproducible: CI (<code>npm ci</code>) installs <i>exactly</i> what was tested locally, not “whatever satisfies the version range today.”
<code>astro.config.mjs</code>	Site config: <code>output: 'static'</code> , the GitHub Pages <code>site/base</code> URL.	Astro needs to know it's deploying to a subpath (<code>/WebCV/</code>) rather than a domain root, so every internal link and asset path resolves correctly.
<code>tsconfig.json</code>	TypeScript compiler config, extending Astro's strict preset.	Turns on type-checking across <code>.ts/.astro</code> files — catches a misspelled YAML field or wrong prop type before it ships.
<code>.gitignore</code>	Excludes <code>dist/</code> , <code>.astro/</code> , <code>node_modules/</code> , <code>.env*</code> from version control.	Build output and dependencies are regenerable and often huge; they don't belong in git history.
<code>.github/workflows/deploy.yml</code>	GitHub Actions CI/CD pipeline: build then deploy on every push to <code>main</code> .	Turns “publish a change” into “git push” — see §7.
<code>content/experience.yaml</code>	Structured list of every work entry (internships, roles).	The actual data source for the Experience page — see §4.
<code>content/projects.yaml</code>	Structured list of every project (title, summary, tags, links, featured flag).	The actual data source for the Projects grid and each project's detail page.
<code>content/cv.md</code> , <code>content/internships.md</code> , <code>content/projects.md</code> , <code>content/extracurriculars.md</code>	Human-readable prose notes — the raw material the YAML files were distilled from.	Kept as reference/source-of-truth prose; not parsed by the site directly, but useful when writing a new YAML entry from scratch.
<code>content/*.pdf</code>	Original source documents (CV versions, project/internship reports).	Kept in <code>content/</code> deliberately <i>not</i> web-accessible — private working material, distinct from the specific PDFs chosen to be publicly linked (copied into <code>public/documents/</code> instead).
<code>public/favicon.svg</code>	Browser-tab icon.	Small brand touch; SVG keeps it crisp at any size with no image processing needed.
<code>public/resume.pdf</code>	The downloadable resume, linked from the hero and Contact section.	Recruiters expect a one-click PDF, not just a webpage, per <code>SPEC.md</code> 's acceptance criteria.

Path	What it is	Why it's needed
<code>public/documents/ *.pdf</code>	Report/slide PDFs linked from specific project entries.	Lives in <code>public/</code> (not <code>content/</code>) specifically because it <i>must</i> be web-accessible — see the <code>content/</code> vs. <code>public/</code> distinction below, and Bug 2 in §8.
<code>public/images/ {experience,projects}/</code>	Org logos / project thumbnails, referenced by optional YAML fields.	Optional visual polish per entry, without forcing every entry to have one (fields are nullable).
<code>public/images/ profile.png</code>	Hero profile photo.	Referenced directly by <code>index.astro</code> ; fails gracefully (hides itself, no broken-image icon) if absent, so the site never looks broken while this is still being decided.
<code>src/lib/content.ts</code>	The data layer: reads/parses/-types <code>content/*.yaml</code> .	The <i>only</i> place YAML is touched — single source of truth for “what an experience/project entry looks like.” See §6.
<code>src/styles/global.css</code>	Design tokens (CSS custom properties), dark mode, responsive base styles.	One place to define color/spacing <i>by role</i> ; every component reuses the same tokens instead of hardcoding values — see §6.
<code>src/components/ Layout.astro</code>	Page shell: <code><html>/<head></code> boilerplate, meta tags, Open Graph tags, nav, footer, <code><slot /></code> .	Written once, so every page gets consistent SEO/meta handling instead of duplicating <code><head></code> content five times.
<code>src/components/ Nav.astro</code>	Site navigation, active-page highlighting.	Builds links from <code>import.meta.env.BASE_URL</code> rather than a hardcoded path, so it keeps working if the deploy target changes.
<code>src/components/ Footer.astro</code>	Site footer.	Shared chrome, same reasoning as <code>Layout.astro</code> .
<code>src/components/ ExperienceCard.astro, ProjectCard.astro</code>	Render one experience/project entry as a card.	Take one typed entry as a prop and know nothing about <i>how many</i> times they're used — the page does the looping (§4).
<code>src/pages/index.astro</code>	The homepage: hero, about, skills, highlights, extracurriculars, contact.	Astro's file-based routing — this file <i>is</i> the route <code>/</code> .
<code>src/pages/ experience.astro</code>	The <code>/experience</code> page: loops over <code>getExperience()</code> .	One template, however many entries currently exist — see §6.
<code>src/pages/projects/ index.astro</code>	The <code>/projects</code> grid page.	Same pattern as Experience, applied to <code>getProjects()</code> .

Path	What it is	Why it's needed
<code>src/pages/projects/[slug].astro</code>	Dynamic route: one generated page per project.	Uses <code>getStaticPaths()</code> so a new YAML entry becomes a new live page with zero route registration — the direct mechanism behind “one YAML entry = one live page.”

5.1 `content/` vs. `public/`: Why Both Exist

They look similar (both hold “data”) but serve opposite purposes:

- **`public/`** is copied *verbatim* into the deployed site — anything here gets its own URL (`public/resume.pdf` → `/WebCV/resume.pdf`).
- **`content/`** is *never* copied to the deployed site — it's read at build time by `content.ts`, turned into HTML, and the original files stay private to the repo.

This is deliberate: raw PDFs and prose notes in `content/` don't need to be individually web-accessible; only the specific documents chosen to link are (copied into `public/documents/`). Mixing these two up caused one of the two real bugs in §8.

6 Code Walkthrough

6.1 `src/lib/content.ts` — the Data Layer

```
function loadYamlList<T>(filename: string): T[] {
  const filePath = path.join(CONTENT_DIR, filename);
  const raw = fs.readFileSync(filePath, 'utf8');
  const data = yaml.load(raw);
  if (!Array.isArray(data)) {
    throw new Error(`${filename} must parse to a YAML list at the top level`);
  }
  return data as T[];
}
```

Plain Node.js: read a file as text, parse it, sanity-check the shape — fail loudly at build time if someone breaks the YAML, rather than silently rendering a blank page. `getExperience()` and `getProjects()` wrap this with the specific filename and a TypeScript type, so every page that imports them gets autocomplete and type errors if a field is misspelled.

`getProjects()` also does the one piece of real logic in this file: sorting — featured projects first, then newest-by-date. That logic lives here once, instead of being duplicated in every page that lists projects (the same “single source of truth” principle from §4, applied to *behavior* instead of *data*).

`formatMonthYear()` turns “2026-07” into “Jul 2026” for display, using the built-in Intl-backed `toLocaleDateString`, so dates are stored in a sortable/parseable machine format in the YAML but shown in a human-readable format on the page.

6.2 `src/components/` — Layout, Nav, Footer, Cards

- **`Layout.astro`** is the page shell every page wraps itself in: emits `<html>/<head>` boilerplate once (meta tags, favicon, Open Graph tags for link previews, canonical URL for SEO), then renders `<Nav>`, `<slot />` (the page's own content), then `<Footer>`.

- **Nav.astro** builds its links from `import.meta.env.BASE_URL` rather than hardcoding `/WebCV/...`, so the same code works whether the site is deployed at a subpath or a domain root. It marks the current page with `aria-current="page"` — an accessibility attribute screen readers announce, which is also what the CSS uses to draw the orange underline.
- **ExperienceCard.astro** / **ProjectCard.astro** each take one entry as a prop and render it, with no knowledge of how many times they're used — the pages loop and call them once per entry.

6.3 src/pages/ — File-Based Routing and the Dynamic-Route Trick

In Astro, the file path *is* the URL — no separate router config. `src/pages/experience.astro` is served at `/experience`.

The interesting one is `src/pages/projects/[slug].astro` — square brackets mean “dynamic route; the value in `[slug]` comes from data, not a fixed filename.” It exports `getStaticPaths()`:

```
export function getStaticPaths() {
  return getProjects().map((project) => ({
    params: { slug: project.slug },
    props: { project },
  }));
}
```

At build time, Astro calls this once, gets back a list (currently 2 items — `verifythevector` and `adversarial-robustness`), and generates one actual `.html` file per item. Add a third project to `projects.yaml` and the *next build* produces a third HTML file automatically — no route to register, because the route was never “`projects/verifythevector`,” it was always “however many projects exist.”

6.4 src/styles/global.css — Design Tokens, Dark Mode, Responsiveness

The palette (a three-anime blend — Naruto orange primary accent, navy/cream base pair, sparing One Piece gold / Food Wars crimson accents; exact colors and reasoning in `SPEC.md §9.1`) is implemented as CSS custom properties, not hardcoded hex values scattered through the stylesheet:

```
:root {
  --color-accent: #e8611c;
  --color-bg: #fbf6ee;
  --color-text: #232323;
  /* ...more tokens... */
}

@media (prefers-color-scheme: dark) {
  :root {
    --color-bg: #1b1f2a;
    --color-text: #edeae3;
    /* accent orange is NOT redefined -- stays the same in both modes */
  }
}
```

Every component then uses `var(--color-bg)` etc. instead of a literal color — a **design-token system**: define each color/size once, by role (“background,” “accent,” “border”) rather than by value, then reference the role everywhere. The payoff: dark mode is a ~10-line override block, not a second copy of every color rule, because only the *tokens* change; every rule using `var(--color-bg)` picks up the new value automatically. This also respects the visitor’s OS-level

light/dark preference automatically via the `prefers-color-scheme` media query, with no toggle button or JavaScript required.

Responsiveness follows the same “define once” instinct: layout uses CSS Grid/Flexbox with relative sizing (e.g. `repeat(auto-fill, minmax(280px, 1fr))`) for card grids — “as many 280px+ columns as fit,” reflowing from multi-column on desktop to single-column on phones with no media query needed) plus two breakpoints (480px, 768px) for the handful of things that need explicit adjustment. `clamp()` is used for heading sizes so text scales smoothly instead of jumping at breakpoints.

7 CI/CD: How a git push Becomes a Live Website

CI/CD (Continuous Integration / Continuous Deployment) means a pipeline runs automatically and builds/publishes for you, consistently, every time — instead of manually running a build and manually uploading files. Here that’s a **GitHub Actions workflow**, defined in `.github/workflows/deploy.yml`.

```
on:
  push:
    branches: [main]      # runs automatically on every push to main
    workflow_dispatch:    # ...and can also be triggered manually

permissions:
  contents: read
  pages: write
  id-token: write         # OIDC auth to Pages, no manual secret needed

jobs:
  build:
    steps:
      - checkout          # clone the repo into the runner
      - setup-node         # install Node.js
      - npm ci             # install deps from the lockfile exactly
      - npm run build      # -> produces dist/
      - upload-pages-artifact # hand dist/ off to the next job

  deploy:
    needs: build          # doesn't start until build succeeds
    steps:
      - deploy-pages      # publish the artifact to GitHub Pages
```

Two jobs, not one, is a deliberate GitHub Pages convention: `build` produces the static files and can run with narrow permissions; `deploy` is the only step that needs permission to publish, and it only runs `needs: build` — if the build fails (a broken YAML file, a typo in a component), `deploy` never runs and the previously-published site stays live. Nothing broken ever gets automatically published.

Why it matters

`npm ci` (not `npm install`) matters here: it installs *exactly* the versions pinned in `package-lock.json`, so CI uses the identical dependency versions as local development — reproducible builds, no “works on my machine.” The `concurrency` block (`group: pages`, `cancel-in-progress: true`) means if two pushes land close together, the older in-flight `deploy` gets cancelled rather than both racing to publish — the site always ends up matching the latest commit.

GitHub Pages itself is a static file host built into GitHub — free, and pointed at “GitHub Actions” as its source (as opposed to “a specific branch’s raw files,” the older Pages model), which is what lets this custom build step run at all.

8 Two Real Bugs Hit and Fixed

Small, concrete, and each shows a debugging *process*, not just a fix — good material for “tell me about a bug you debugged.”

8.1 Bug 1: Broken Internal Links (/WebCVexperience)

Symptom: after the first successful build, every internal link was missing a slash — `href="/WebCVexperience"` instead of `href="/WebCV/experience"`.

Root cause: `astro.config.mjs` had `base: '/WebCV'` (no trailing slash). Astro exposes that value unmodified as `import.meta.env.BASE_URL`. Every link in the codebase was built as ``${base}experience`` — string concatenation, expecting `base` to already end in `/`. Without the trailing slash, that produced `"/WebCV" + "experience"`.

How it was found: not by staring at code — by checking the actual build output. After `astro build`, every generated `href="..."` was grepped and read. The build itself didn’t error (concatenating two strings is valid JS), so this was a “silently wrong output” bug — the kind that’s easy to ship if you only check “did the build succeed” instead of “is the output actually correct.”

Fix: one character — `base: '/WebCV/'` — plus rebuilding and re-grepping to confirm every link was correct, with a comment left on that line explaining why the trailing slash is required.

8.2 Bug 2: Report/Slide PDF Links Pointed at Files That Don’t Exist in the Deployed Site

Symptom: `projects.yaml` originally had `report: "/content/Final_Report_VerifyTheVector.pdf"`. That path is valid *in the repo*, but `content/` is never copied into `dist/` (§5) — so that link would 404 on the live site, even though nothing about the build failed.

How it was found: same instinct as Bug 1 — after building, what got written to `dist/` was actually inspected, tracing whether every linked file existed there.

Fix: the three linked PDFs were copied into `public/documents/` (the folder that *does* get deployed) and the YAML paths updated to point there, plus the page template updated to prefix internal paths with the base path (same mechanism as Bug 1), leaving external links (GitHub repo URLs) untouched.

Why it matters

A successful build (exit code 0) proves the code *compiled*, not that the output is *correct*. Verifying this project meant actually reading the generated HTML and following the links, not just trusting a green checkmark.

9 Design-Process Decisions Worth Explaining

- **Why write SPEC.md before writing any code?** To separate “what does this need to do” from “how do I build it,” and to have a written artifact of requirements (acceptance criteria, breakpoints, color palette) that can be checked against later instead of relying on memory

— standard practice on real engineering teams: a design doc precedes implementation so decisions get made deliberately, not accidentally while typing code.

- **Why `PLAN.md` in addition to `SPEC.md`?** `SPEC.md` answers “what must be true when this is done” and barely changes. `PLAN.md` answers “what’s actually done right now, and what did we decide along the way” — it changes constantly and is where decisions (“use Astro,” “use GitHub Pages,” “this repo is private, don’t link it”) get written down *once*.
- **Why per-project detail pages instead of just a grid?** A single grid page has to compress every project into a two-line summary. Once there’s more than a couple of projects, a dedicated page per project lets each one carry its full writeup, tech stack, and links without cluttering the overview — and gives each project its own shareable URL.

10 Glossary

Term	Meaning
Static site generation (SSG)	Building HTML files once, ahead of time, rather than per-request on a server (SSR) or in the browser via JavaScript (client-side rendering/SPA).
Build time vs. runtime	“Build time” = when <code>astro build</code> runs, in CI, before anyone visits. “Runtime” = when a real visitor’s browser loads the page. This project does essentially nothing at runtime beyond serving static files.
CI/CD	Continuous Integration / Continuous Deployment — automatically building and publishing on every change, instead of doing it by hand.
YAML	A human-readable data format (like JSON, but supports comments and is less punctuation-heavy) — used for <code>content/*.yaml</code> .
Design tokens	Named variables for design values (colors, spacing) defined once and referenced everywhere, so changing the palette means editing one place.
<code>prefers-color-scheme</code>	A CSS media query that detects the visitor’s OS-level light/-dark mode preference.
OIDC	A way for a CI job to prove its identity to a service (here, GitHub Pages) without a manually-created, manually-rotated secret token.
<code>getStaticPaths</code>	An Astro function that says “generate one page per item in this list” for a dynamic route — the mechanism behind <code>/projects/[slug].astro</code> producing one real HTML file per project.
Base path	The URL prefix a site is served under when it’s not at a domain’s root (here, <code>/WebCV/</code> , because GitHub Pages project sites live at <code>username.github.io/repo-name/</code>).

11 Interview Cheat-Sheet

“Walk me through this project.”

Use the pitch in §1, then let them steer with follow-ups.

“Why Astro instead of React/Next.js?”

The content barely changes and there’s no interactivity that needs a client-side framework; Astro renders to plain HTML at build time so nothing ships to the browser except what’s actually needed, while still giving reusable components and data-driven templating.

“How would you add a new project to the site?”

Copy the template block in `content/projects.yaml`, fill in the fields, save. Next build/deploy produces a new card on `/projects` and a whole new detail page automatically — no code change.

“What happens when you push to main?”

Walk through §7: a build job (`npm ci`, `astro build`), then a `deploy` job that only runs if build succeeded, publishing to GitHub Pages. Live in under a minute.

“Tell me about a bug you ran into.”

Either bug in §8 — both have a clear symptom, root cause, and fix, and both illustrate the same lesson: a successful build doesn’t guarantee correct output.

“How is content separated from code?”

§4: YAML files hold data, a loader reads and types it, page templates loop over whatever the loader returns.

“How does dark mode work?”

§6: CSS custom properties define colors by role; a `prefers-color-scheme: dark` media query overrides just those variables; every rule referencing a variable updates automatically — no JavaScript, no toggle.

“Why didn’t you use a database / CMS?”

The content is small, changes rarely, and there’s already a source of truth (plain text files in git). A database/CMS adds infrastructure and a network dependency to solve a problem version-controlled files already solve for free.